

Návrh implementácie vybraných algoritmov riadenia

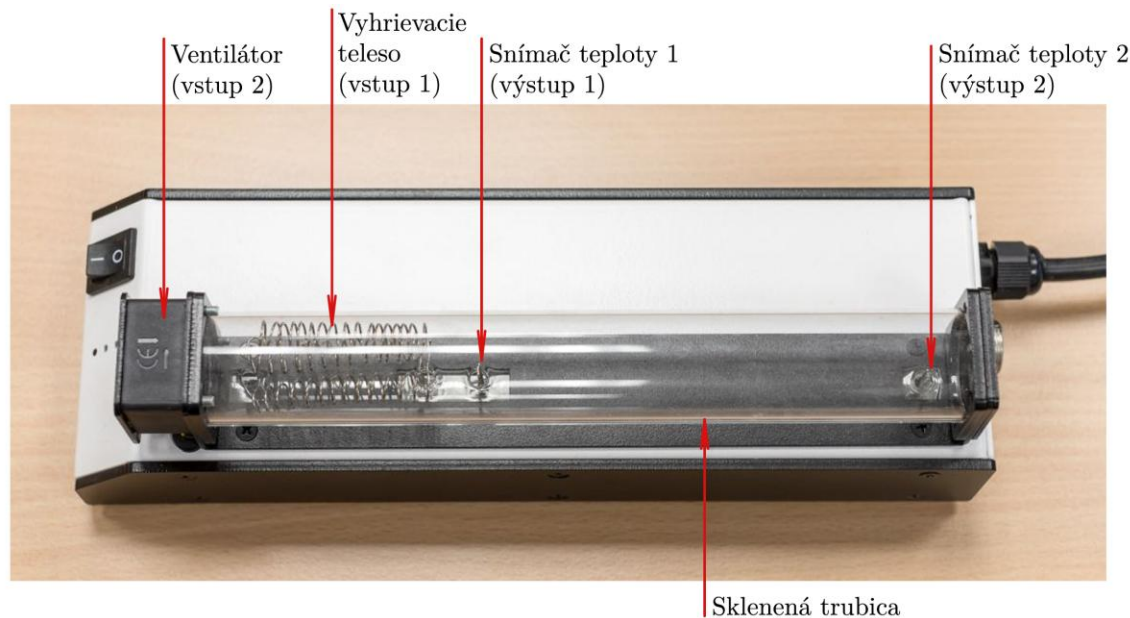
Diplomová práca

Bc. Andrii Stoliar
Ing. Marián Tárník, PhD.

Ciele práce

- Oboznámte sa s dostupným laboratórnym hardvérom a vypracujte stručnú technickú dokumentáciu vybraných laboratórných dynamických systémov.
- Vypracujte krátky prehľad riadiacich algoritmov vybraných pre následnú implementáciu.
- Navrhnite a realizujte predmetné riadiace systémy a analyzujte kvalitu riadenia.
- Vyhodnot'te dosiahnuté výsledky.

Laboratórne zariadenie TS - popis systému

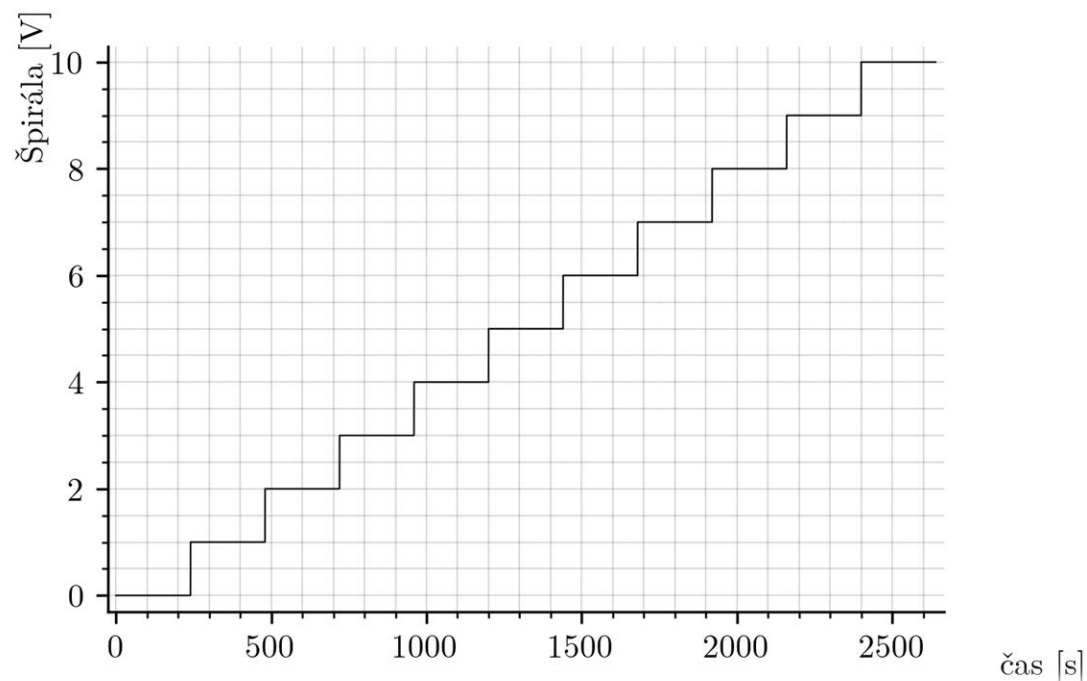


Obr. 1: Laboratórny model TS05

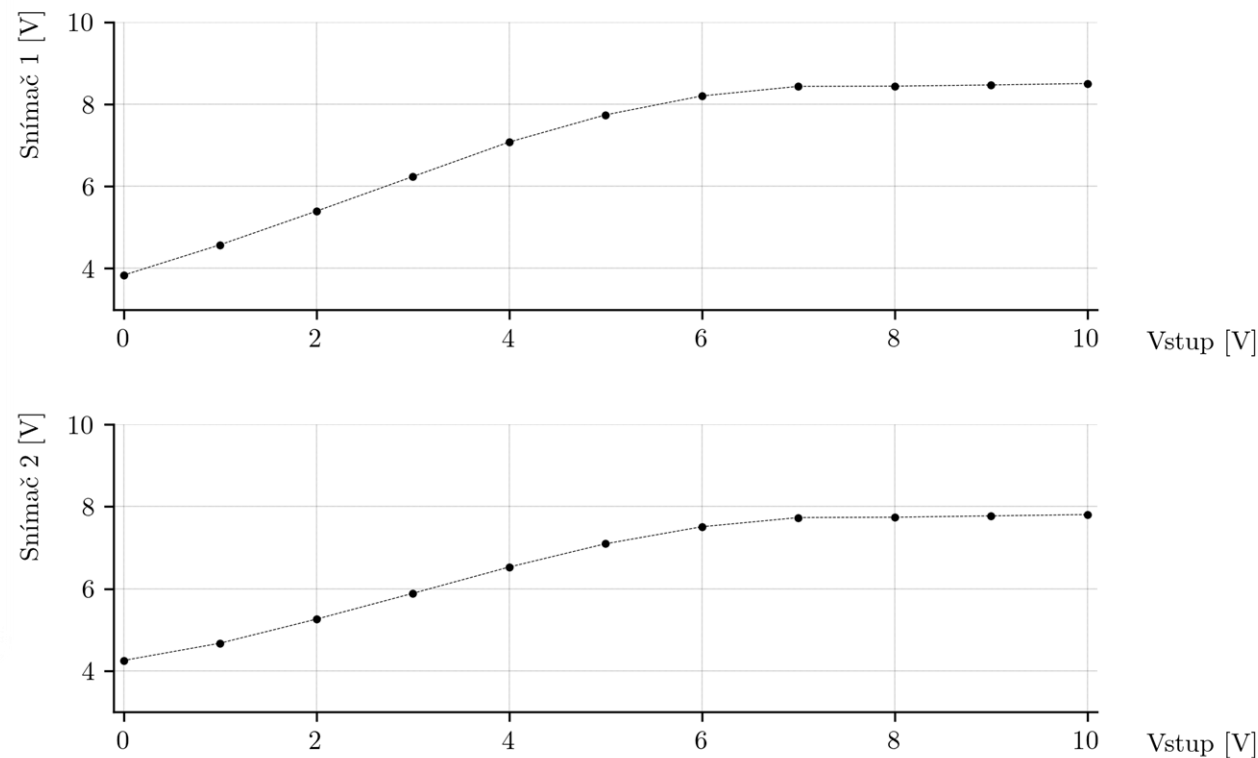
Tabuľka 1: Vstupy a výstupy systému TS05

Typ signálu	Označenie	Rozsah
Vstup	Ventilátor	0–10 V
Vstup	Výkon špirály	0–10 V
Výstup	Senzor 1 (teplota 1)	0–10 V
Výstup	Senzor 2 (teplota 2)	0–10 V

Statické vlastnosti systému

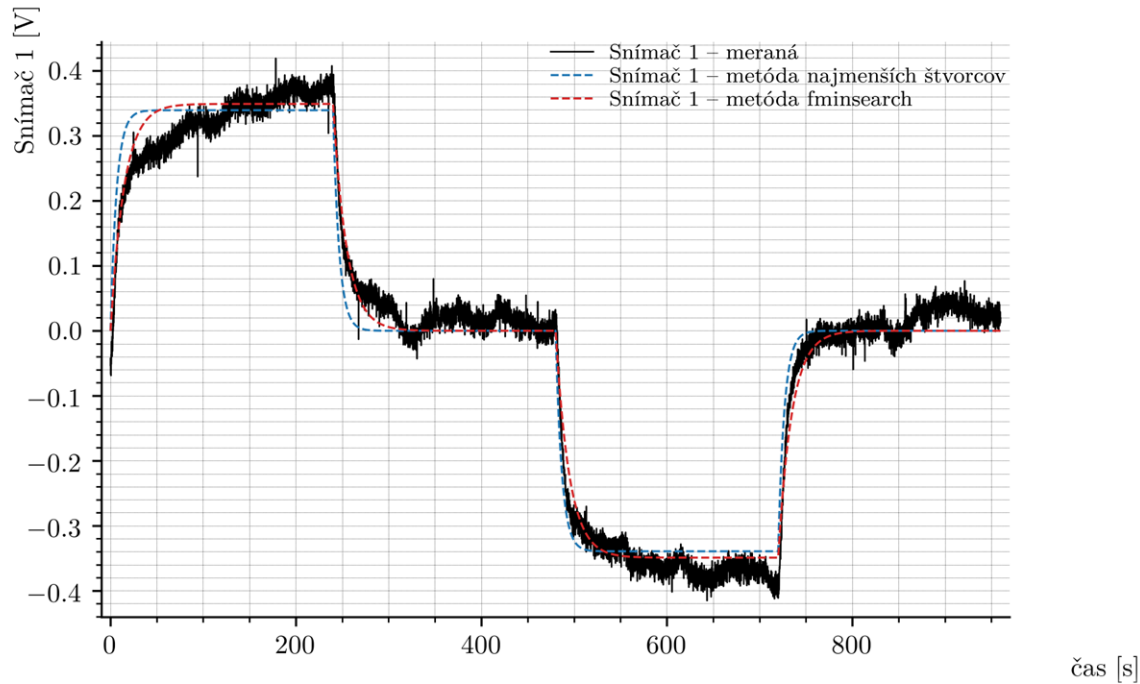


Obr. 3: Priebeh napätia na špirále



Obr. 8: Prevodová charakteristika systému TS05 pre pracovný bod ventilátora 6 V

Identifikácia systému v pracovnom bode: ventilator – 6V, u (špirála) – 4V, $\Delta u = 0.5V$



Obr. 14: Porovnanie skokovej odozvy reálneho systému TS05 a aproximovaných modelov

$$RMSE_{LS} = 0.036792$$
$$RMSE_{fmin} = 0.026996$$

$$G_{s,LS}(s) = \frac{0.00171s^2 - 0.3379s + 6.074}{s^2 + 59.01s + 8.95}$$



$$G_{s,LS}(s) = \frac{6.074}{s^2 + 59.01s + 8.95}$$

$$RMSE_{LS,approx} = 0.036951$$
$$s_1 = -58.855, \quad s_2 = -0.1521$$

Komunikačné rozhranie systému TS05 pre jednotlivé platformy

MATLAB/Simulink

- Komunikácia cez meraciu kartu Advantech PCI-1711
- Použité analógové vstupy/výstupy v Simulinku

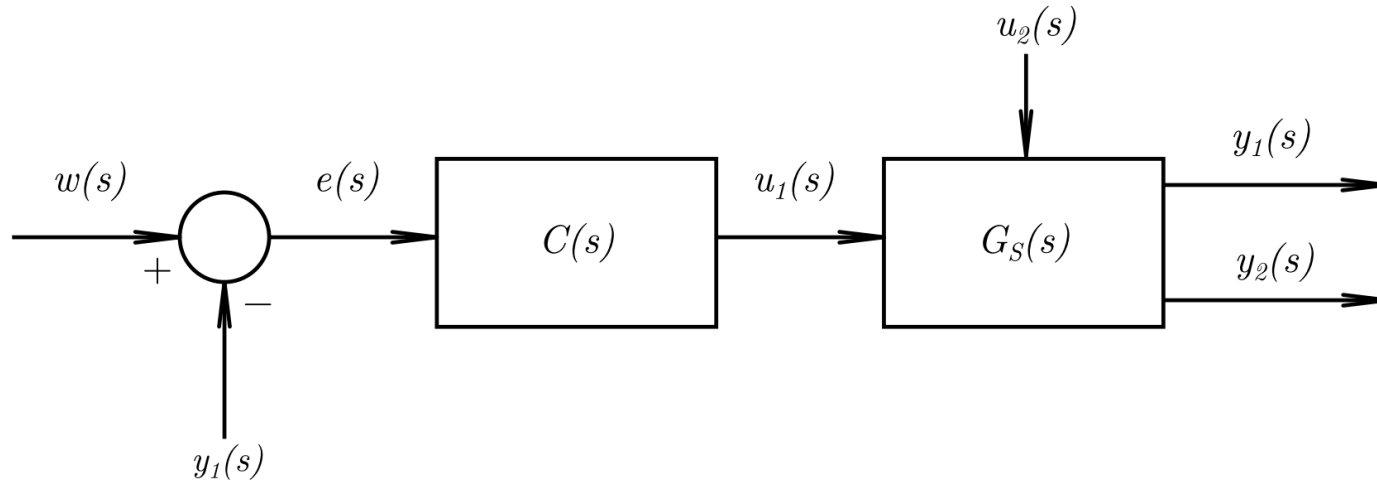
Arduino UNO, C++

- Vytvorená vlastná hardvérová komunikácia s TS05
- Identifikované piny 8-pinového konektora TS05
- Výstupy cez PWM to 0–10 V prevodník
- Snímače 0–10 V upravené napäťovým deličom na 0–5 V
- Použitá kalibrácia signálových ciest kubickými polynómami

PLC Siemens S7-1200 G2

- Priame pripojenie cez analógový modul SM 1233 AI/AQ
- Analógové signály spracované v rozsahu 0–10 V
- Interný rozsah PLC: 0–27648 $\leftrightarrow$ 0–10 V
- Riadiaca logika v bloku Cyclic Interrupt, $T_s = 0,1$ s
- Logovanie realizované do interných polí PLC

PI regulátor pre riadenie v okolí jedného pracovného bodu



Obr. 19: Bloková schéma uzavretého regulačného obvodu s PI regulátorom

Výsledky použitia pidtune():

$$K_p = 3.58, \quad K_i = 1.84$$

$$G_{oro}(s) = \frac{21.21s + 11.15}{s^3 + 59.01s^2 + 8.95s}$$

$$s_1 = 0, \quad s_2 = -58.855, \quad s_3 = -0.1521$$

Paralelná forma PI regulátora:

$$C(s) = K_p + \frac{K_i}{s}$$

Zákon riadenia:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau$$

Diskretizácia:

$$\begin{aligned} e(k) &= w(k) - y(k), \\ I(k) &= I(k-1) + T_s e(k), \\ u(k) &= K_p e(k) + K_i I(k). \end{aligned}$$

Pseudokód a Implementácia MATLAB/Simulink

```
% Inputs: w (setpoint), y (output), Ts  
% States: I (integrator state)  
% Params: Kp, Ki, u_min, u_max
```

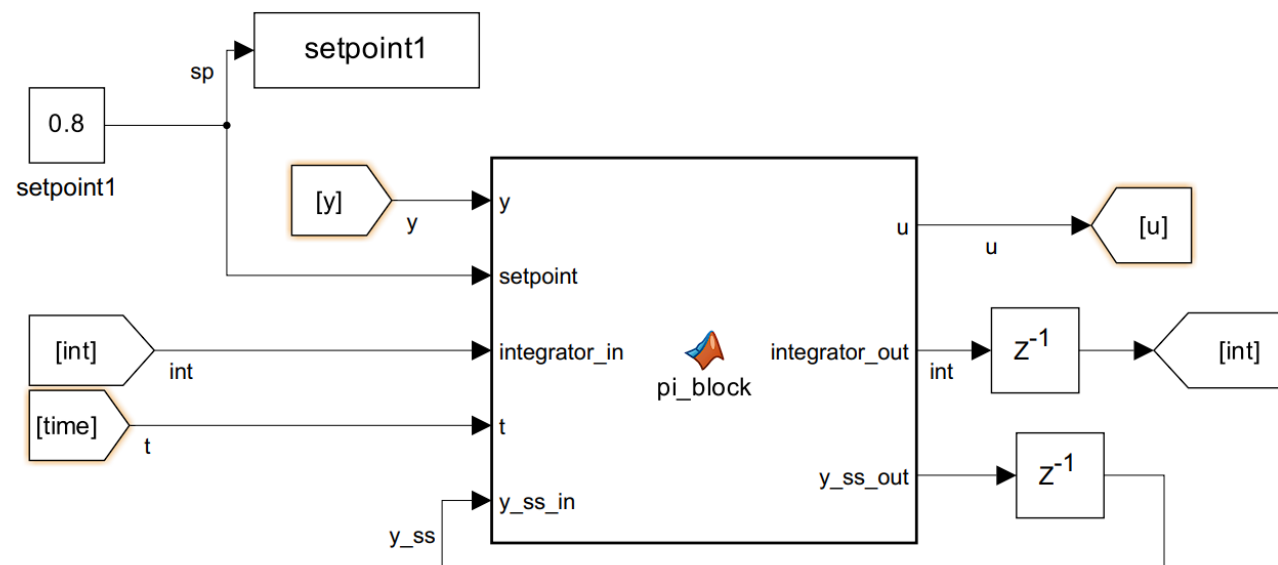
```
e = w - y;
```

```
% Integrator update  
I = I + Ts * e;
```

```
% PI law  
u = Kp * e + Ki * I;
```

```
% Saturation  
u = min(max(u, u_min), u_max);
```

Výpis kódu 8: Pseudokód diskkrétnej realizácie PI regulátora so saturáciou



Obr. 21: Zapojenie PI regulátora v bloku MATLAB Function so spätnou väzbou stavov

Implementácia: mikrokontrolér a jazyk C++

```
float vent = 6.0f;  
float spirala = 0.0f;  
float u_pb = 4.0f;  
float u_pb_min = -u_pb;  
float u_pb_max = 10 - u_pb;  
float Kp = 3.58f;  
float Ki = 1.84;  
float u = 0.0f;  
float y_pb = 0.0f;  
float integrator = 0.0f;  
float e = 0.0f;  
float setpoint_pb = 0.35f;  
float dy = 0.0f;  
float du = 0.0f;
```

Výpis kódu 11: Inicializácia parametrov PI regulátora v jazyku C++

```
if (globalTime < 10 * 60) {  
    u = u_pb;  
    y_pb = snimac1;  
    integrator = 0.0f;  
} else {  
    dy = snimac1 - y_pb;  
    e = setpoint_pb - dy;  
  
    integrator += e * timeTickPeriod;  
  
    du = Kp * e + Ki * integrator;  
    du = clampFloat(du, u_pb_min, u_pb_max);  
  
    u = u_pb + du;  
}  
  
spirala = u;
```

Výpis kódu 12: Hlavná regulačná logika PI regulátora v jazyku C++

aplikácia $u(k-1)$



meranie $y(k)$



výpočet $u(k)$



čakanie do T_s

Implementácia: programovateľný logický automat (PLC)

```

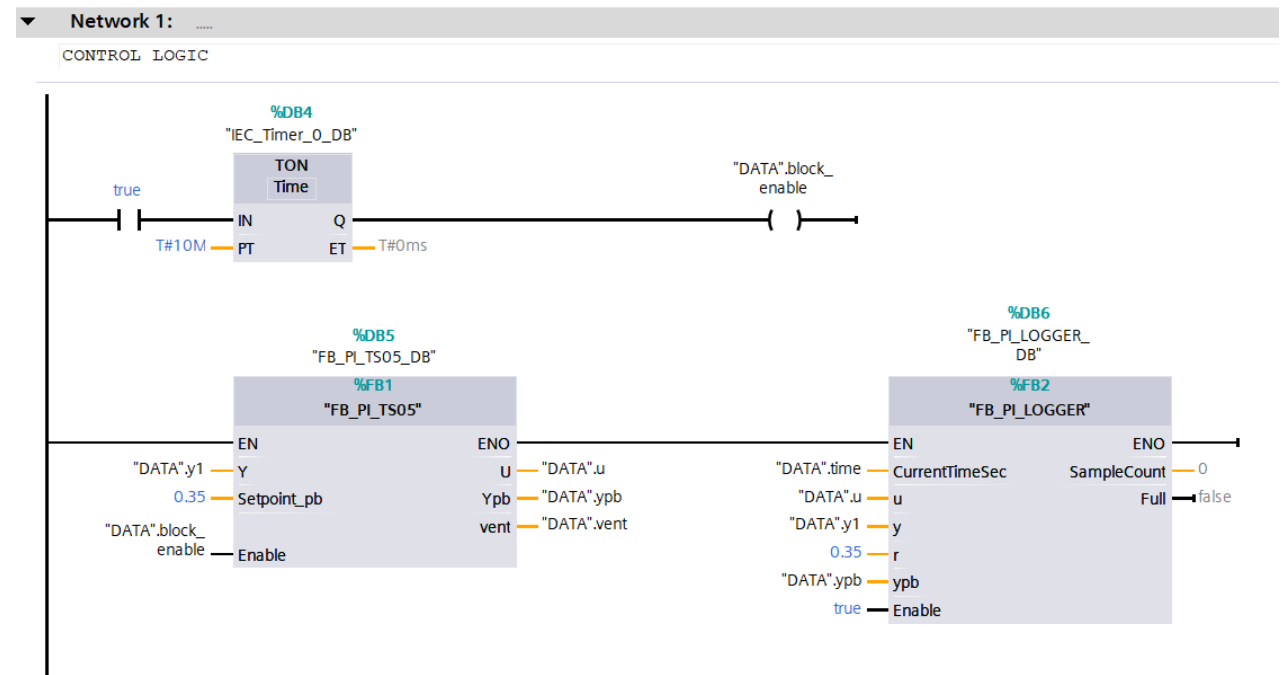
IF NOT #Enable THEN
    #U := #U_pb;
    #DU := 0.0;
    #E := 0.0;
    #Integrator := 0.0;
    #Ypb := #Y;
    #vent := 6.0;
ELSE
    #E := #Setpoint_pb - (#Y - #Ypb);
    #Integrator := #Integrator + #E * #Ts;
    #DU := #Kp * #E + #Ki * #Integrator;

    IF #DU > #DU_max THEN
        #DU := #DU_max;
    ELSIF #DU < #DU_min THEN
        #DU := #DU_min;
    END_IF;

    #U := #U_pb + #DU;
    #vent := 6.0;
END_IF;

```

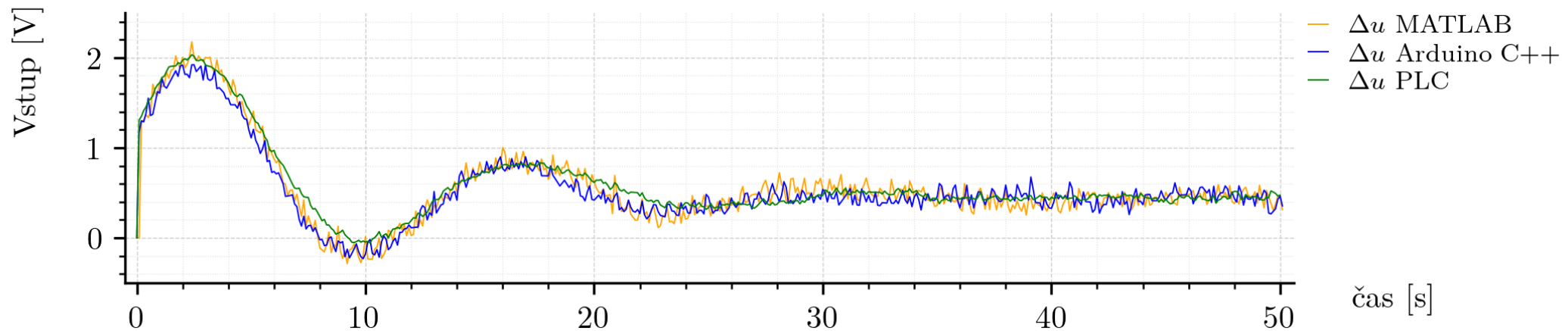
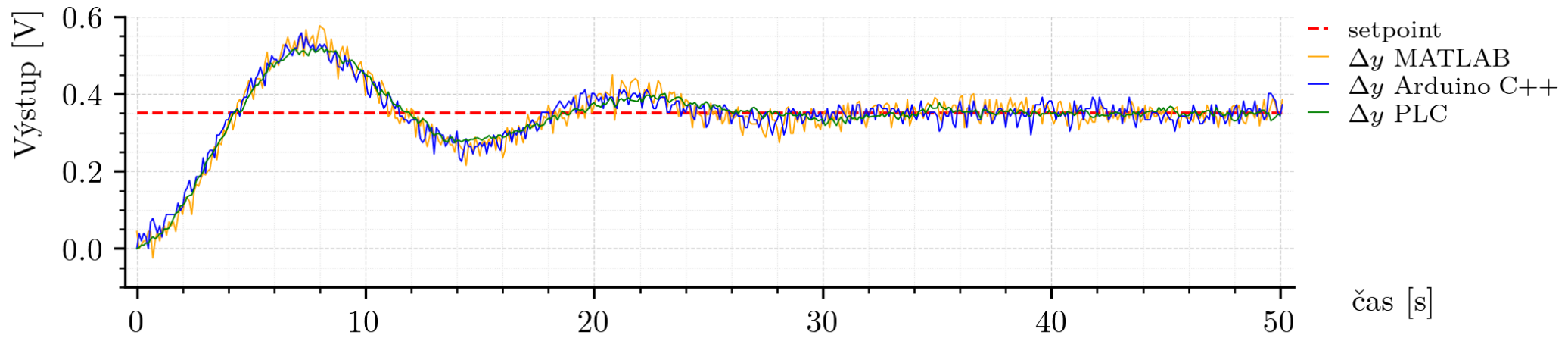
Výpis kódu 13: Implementácia funkčného bloku
FB_PI_TS05 v PLC Siemens



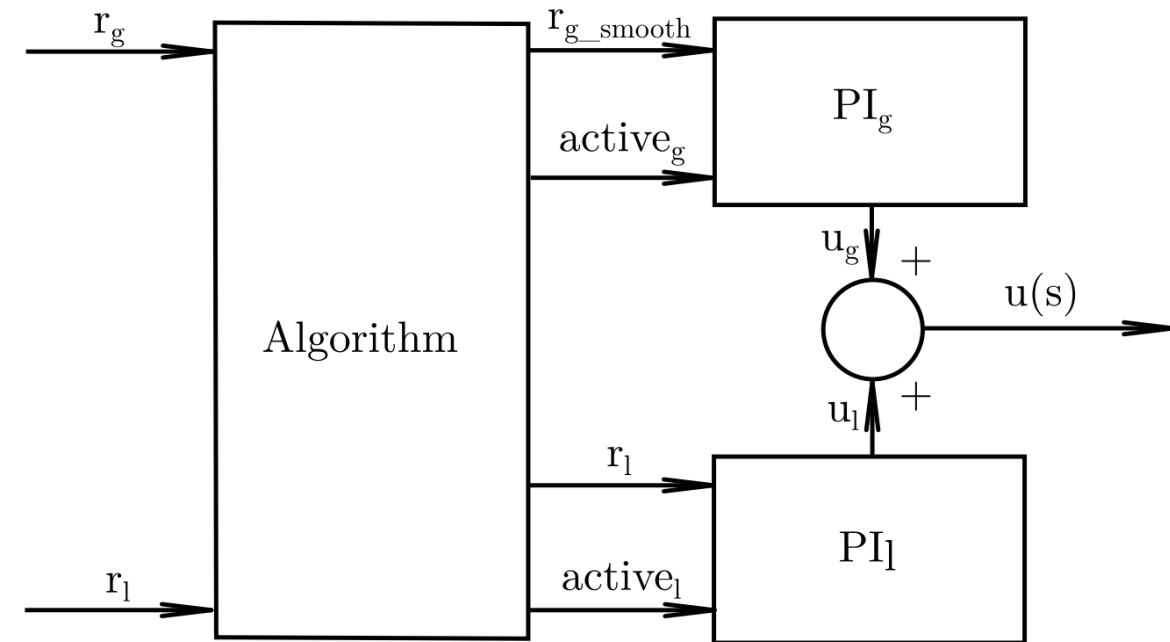
Obr. 26: Zapojenie PI regulátora, časovača a logovania v prostredí TIA Portal

Porovnanie výsledkov z jednotlivých implementácií

$$\begin{aligned}RMSE_{MATLAB} &= 0.092899 \text{ V}, \\RMSE_{C++} &= 0.085426 \text{ V}, \\RMSE_{PLC} &= 0.08576 \text{ V}.\end{aligned}$$



Riadiaci systém s prepínaním pre celý rozsah výstupnej veličiny riadeného systému



Obr. 28: Bloková schéma riadenia s globálnym a lokálnym PI regulátorom

$$\Delta y = y - y_{pb}, \quad \Delta u = u - u_{pb}$$

u_{pb}	y_{pb}	Δy_{pb} pri $u_{pb} \pm 0.5 \text{ V}$	PI parameter
2 V	4.4884	± 0.41	$K_p = 2.77, K_i = 1.48$
4 V	6.3412	± 0.35	$K_p = 3.58, K_i = 1.84$
6 V	7.7441	± 0.18	$K_p = 1.83, K_i = 2.2$

Tabuľka 12: Parametre pracovných bodov, identifikované prenosové funkcie a PI regulátory

Filter pre r_g :

$$G_f(s) = \frac{1}{(10s + 1)^2}$$

Zákon riadenia

Binárne aktivačné signály:

$$a_l, a_g \in \{0, 1\}$$

Chyby regulátorov:

$$e_l = r_l - (y - y_{pb}) = r_l - \Delta y$$

$$e_g = \frac{r_g}{(10s + 1)^2} - y = r_{g,f} - y$$

Zákony riadenia regulátorov:

$$u_l = u_{pb} + K_{p,l}e_l + K_{i,l}I_l$$

$$u_g = K_{p,g}e_g + K_{i,g}I_g$$

Zákon riadenia:

$$u = \begin{cases} u_{pb} + K_{p,l}e_l + K_{i,l}I_l, & \text{ak } a_l = 1, a_g = 0 \\ K_{p,g}e_g + K_{i,g}I_g, & \text{ak } a_l = 0, a_g = 1 \end{cases}$$

Počiatkové stavy integrátorov:

$$I_l = \frac{(u - u_{pb}) - K_{p,l}e_l}{K_{i,l}}, \quad a_l = 0$$

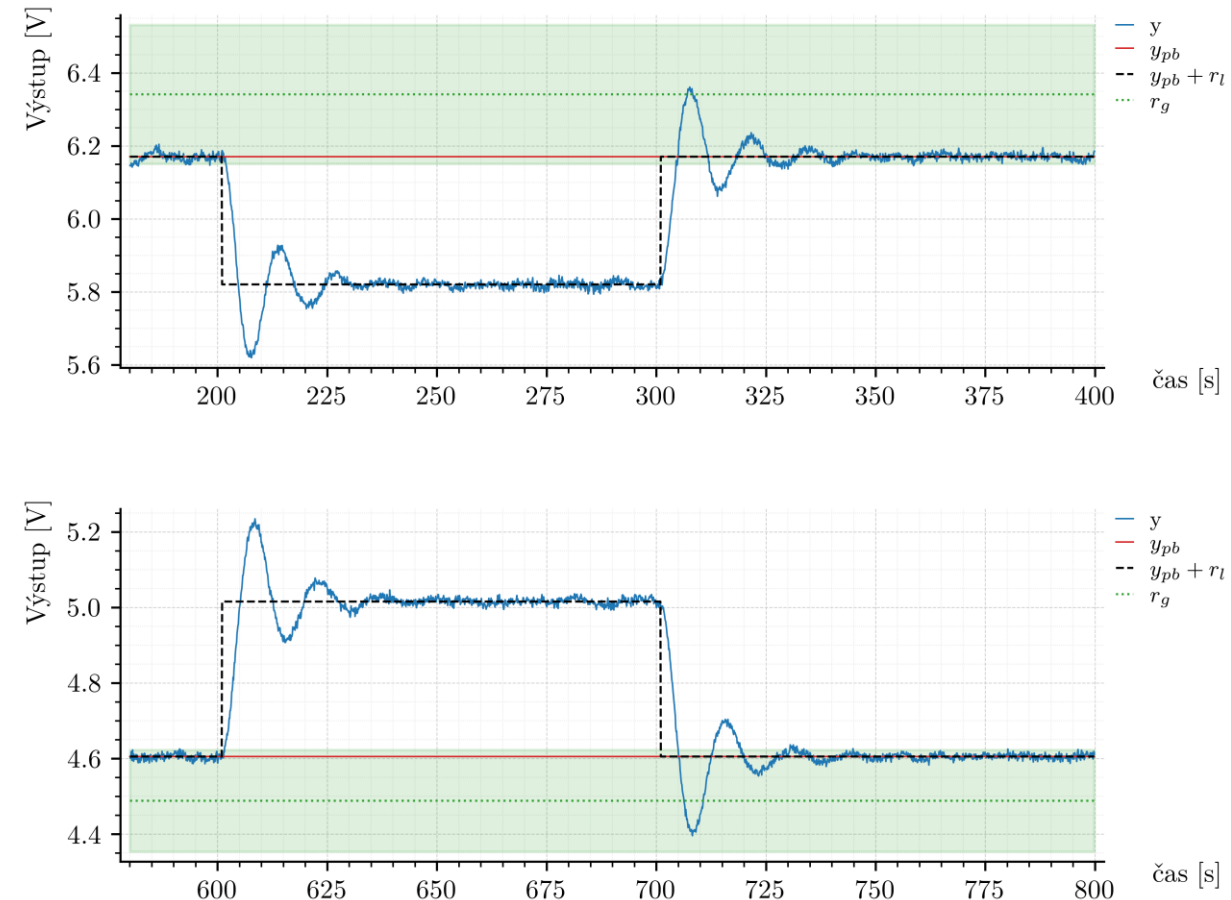
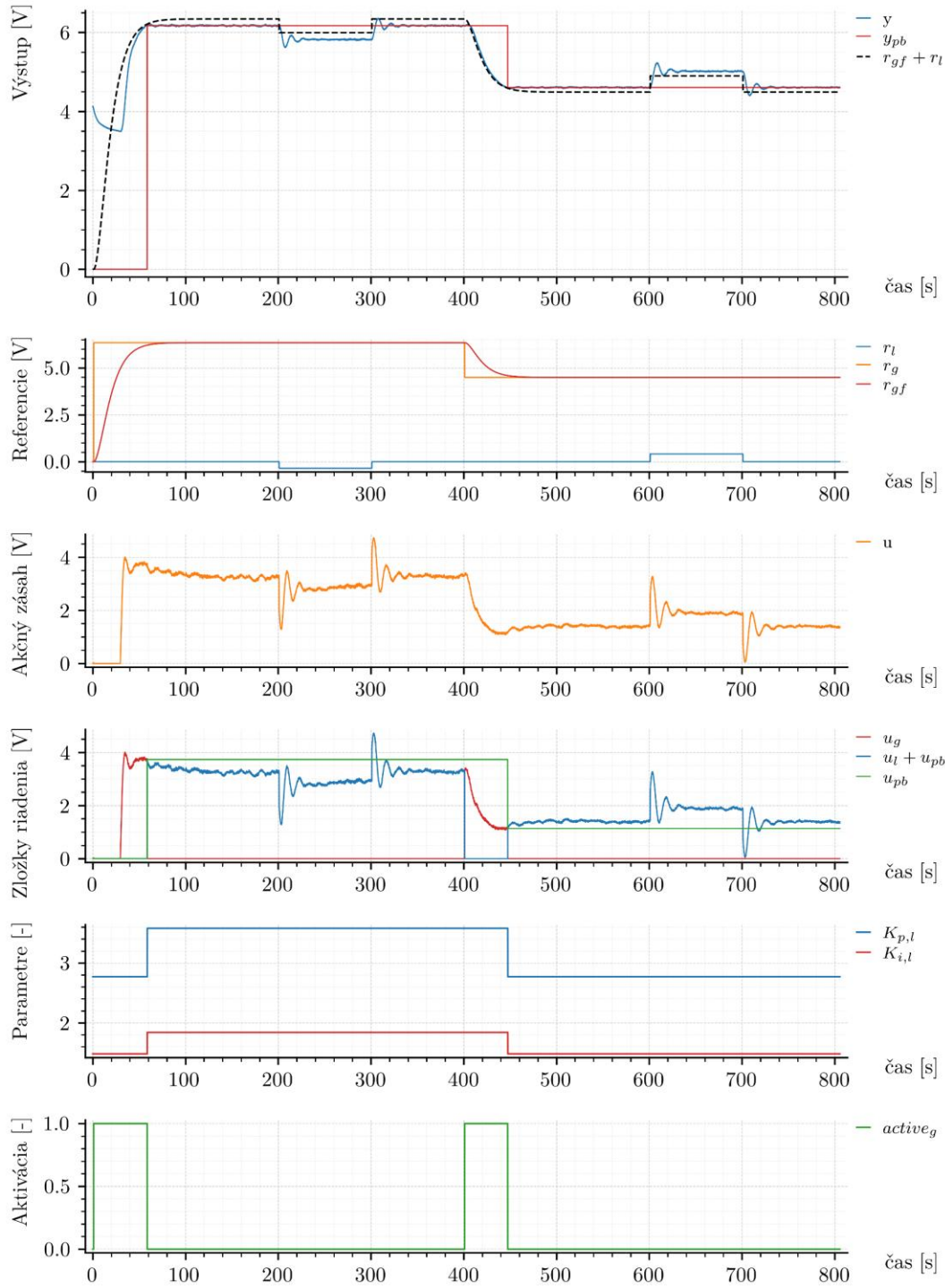
$$I_g = \frac{u - K_{p,g}e_g}{K_{i,g}}, \quad a_g = 0$$

Prepnutie:

Za ustálený stav, keď sa aspoň 25 z posledných 30 hodnôt výstupu nachádza v intervale:

$$\langle r_g - 0.03r_g, \quad r_g + 0.03r_g \rangle$$

$$y_{pb}^* = \frac{1}{30} \sum_{i=1}^{30} y_i, \quad u_{pb}^* = \frac{1}{30} \sum_{i=1}^{30} u_i$$

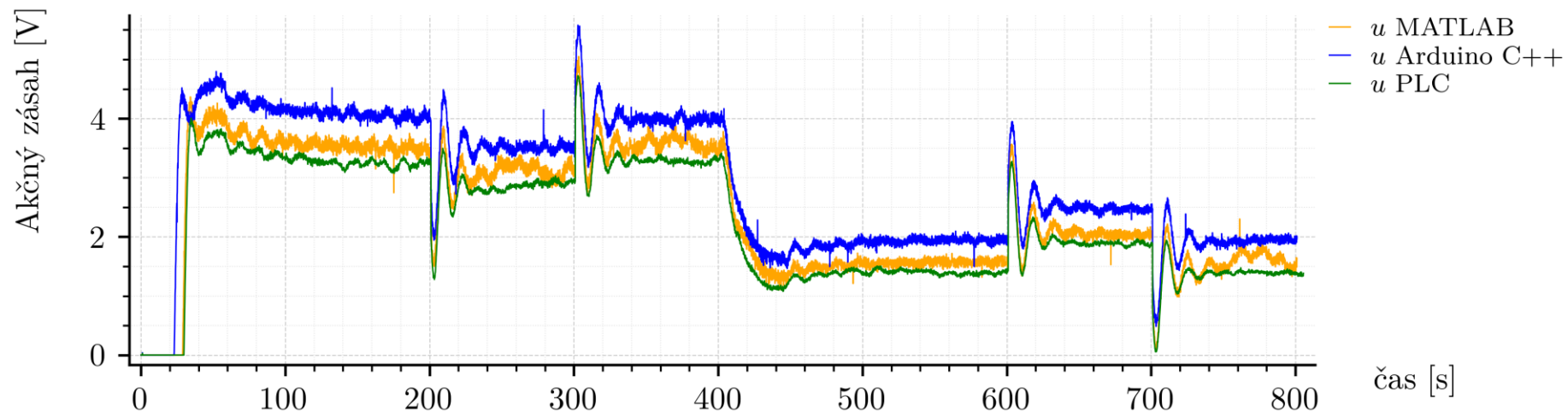
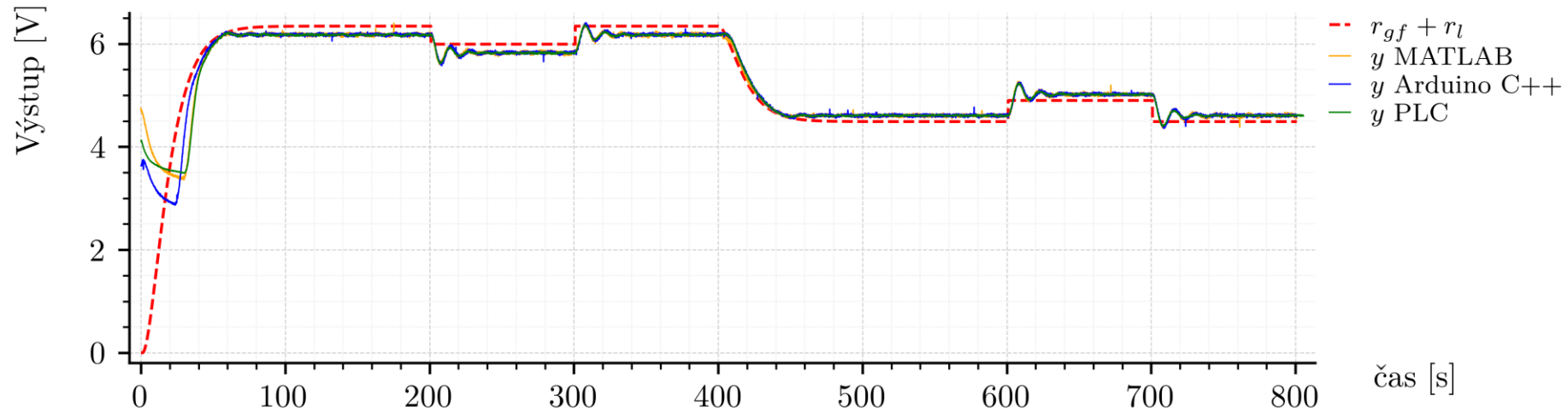


Obr. 37: Detail regulácie v okolí pracovného bodu pri implementácii algoritmu v PLC Siemens

Obr. 36: Experimentálne overenie prepínania pracovných bodov pri implementácii algoritmu v PLC Siemens

Porovnanie výsledkov z jednotlivých implementácií

$$\begin{aligned} RMSE_{MATLAB} &= 0.497536 \text{ V}, \\ RMSE_{C++} &= 0.405534 \text{ V}, \\ RMSE_{PLC} &= 0.456926 \text{ V}. \end{aligned}$$



Adaptívne riadenie s referenčným modelom

Referenčný model:

$$\frac{y_m(s)}{r(s)} = \frac{1}{s^2 + 20s + 1}$$

Prechod do pracovného bodu:

$$\Delta y(k) = y(k) - y_{pb},$$

$$\Delta r(k) = r(k) - r_{pb},$$

$$\Delta u(k) = u(k) - u_{pb}.$$

Numerická derivácia výstupu:

$$\Delta \dot{y}(k) = \frac{\Delta y(k) - \Delta y(k-1)}{T_s}$$

Chyba sledovania modelu:

$$e(k) = \Delta y(k) - y_m(k)$$

Regresný vector:

$$w(k) = \begin{bmatrix} \Delta y(k) \\ \Delta \dot{y}(k) \\ \Delta r(k) \end{bmatrix}$$

Zákon riadenia:

$$\Delta u(k) = \theta^T(k)w(k)$$

$$u(k) = u_{pb} + \Delta u(k)$$

Adaptačný zákon parametrov:

$$\dot{\theta}_1 = -\alpha_1 e * \text{sign}(b_0) \frac{1}{s^2 + a_{1m}s + a_{0m}} y,$$

$$\dot{\theta}_2 = -\alpha_1 e * \text{sign}(b_0) \frac{s}{s^2 + a_{1m}s + a_{0m}} y,$$

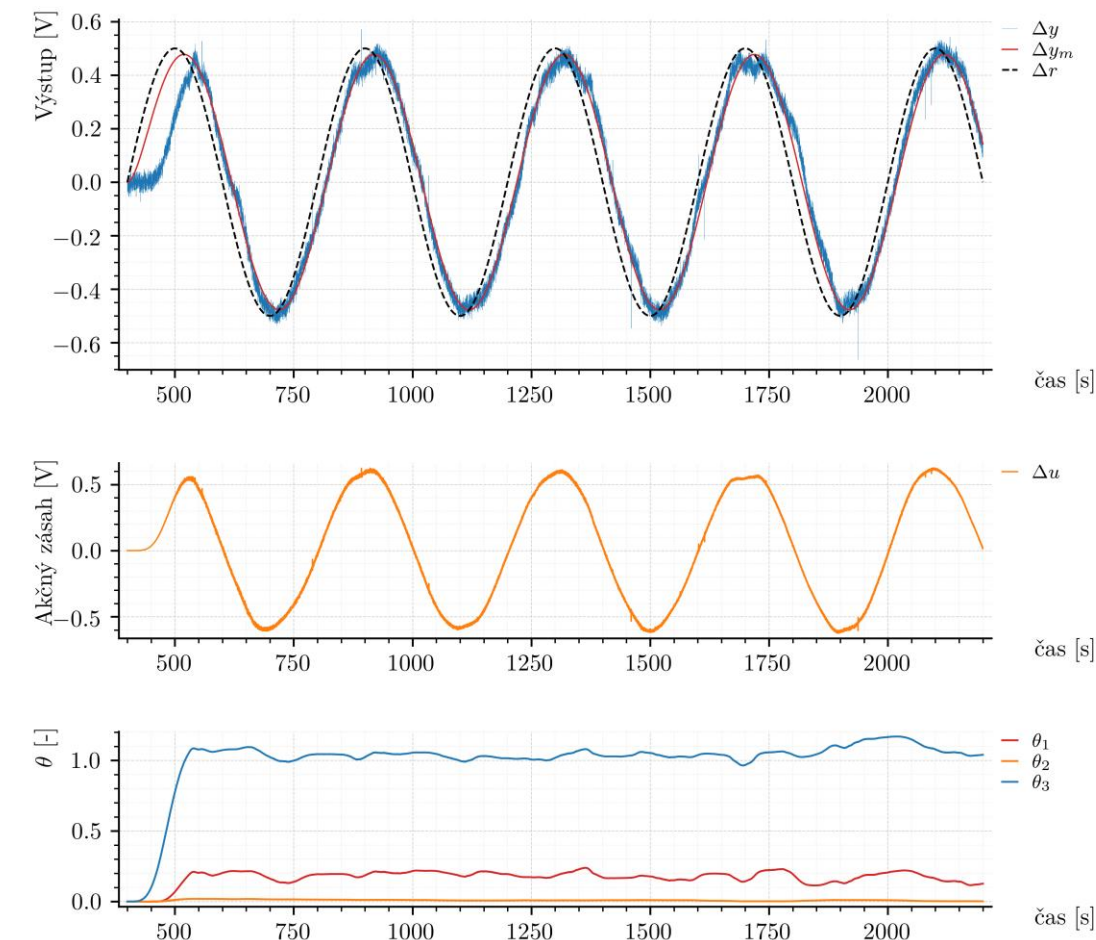
$$\dot{\theta}_3 = -\alpha_1 e * \text{sign}(b_0) \frac{1}{s^2 + a_{1m}s + a_{0m}} r.$$

Diskrétna aktualizácia:

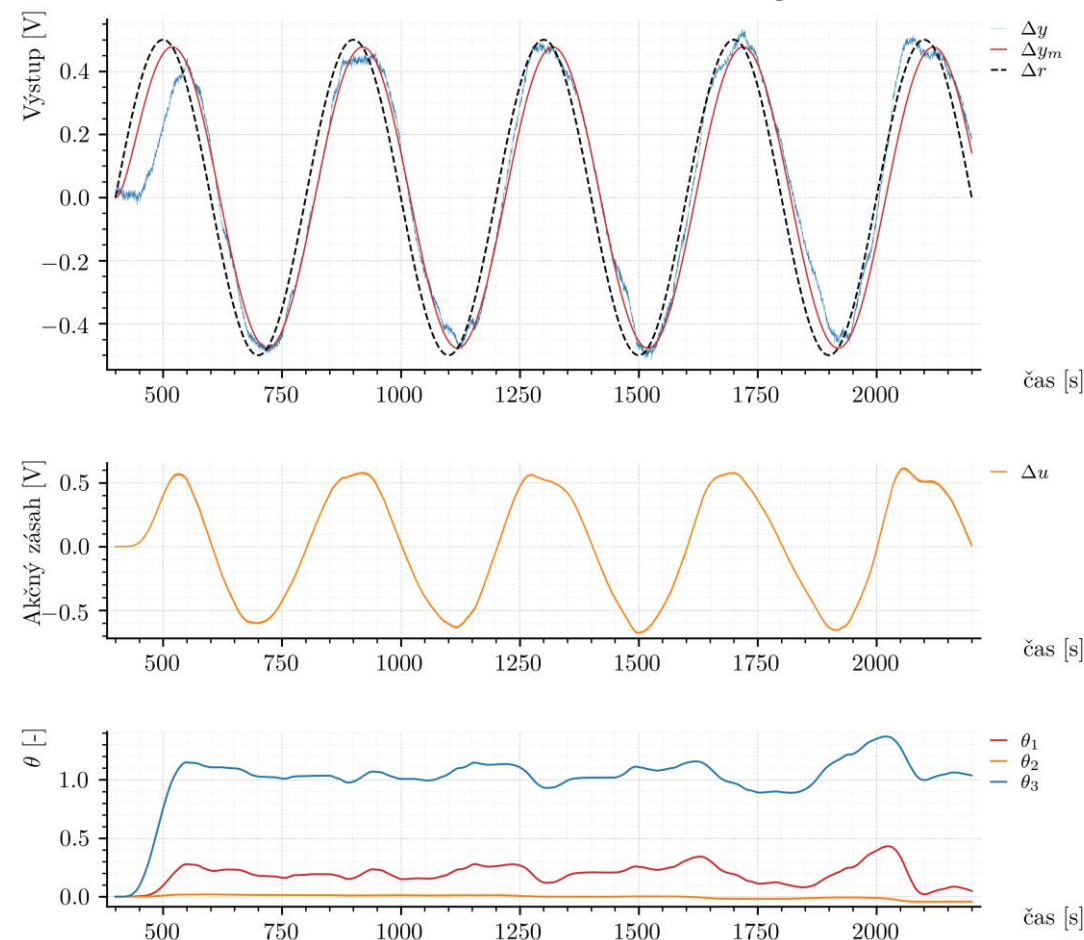
$$\theta(k+1) = \theta(k) + T_s \dot{\theta}(k)$$

Výsledky MATLAB/Simulink a PLC

$$RMSE_{MATLAB} = 0.0621 \text{ V},$$
$$RMSE_{PLC} = 0.0742 \text{ V}.$$

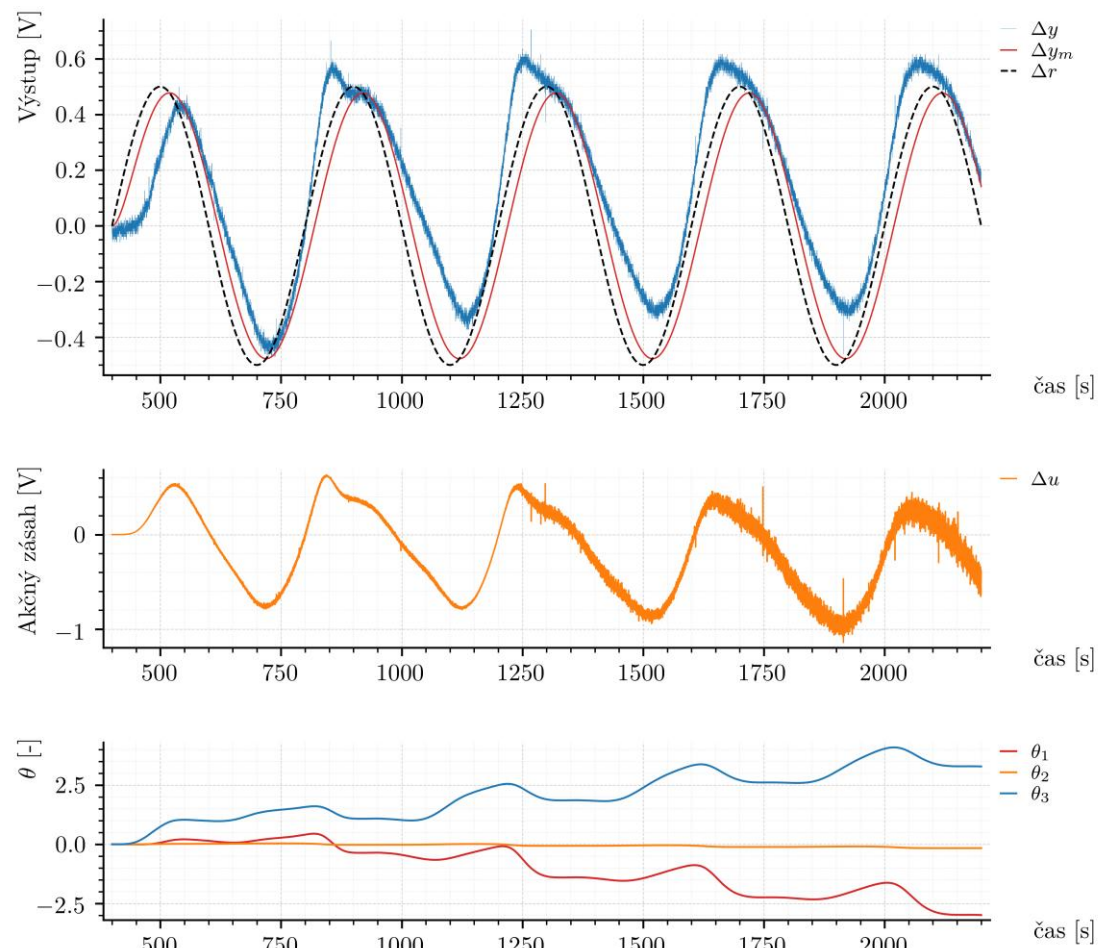


Obr. 41: Experimentálne výsledky gradientného MRAC regulátora MATLAB



Obr. 47: Experimentálne výsledky gradientného MRAC regulátora PLC Siemens

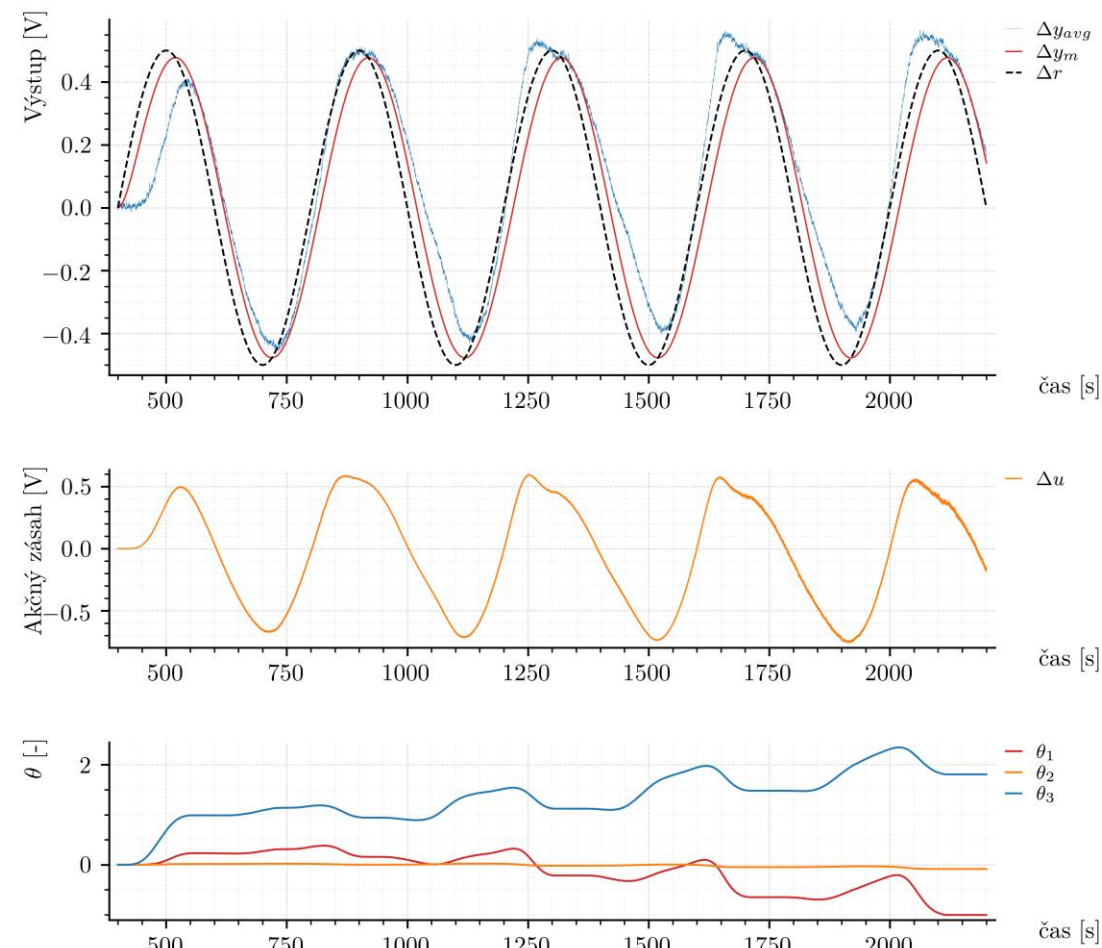
Výsledky Arduino C++



Obr. 43: Experimentálne výsledky gradientného MRAC regulátora v C++

$$RMSE_{C++} = 0.1763 \text{ V},$$

$$RMSE_{C++ex.} = 0.1274 \text{ V}.$$



Obr. 44: Experimentálne výsledky heuristicky upravenej implementácie gradientného MRAC regulátora v C++

Zaver...

- Zdokumentovaný laboratórny systém TS05
- Pripravená komunikácia pre MATLAB/Simulink, Arduino C++ a PLC Siemens
- Implementované tri algoritmy: PI, PI s prepínaním pracovných bodov a MRAC
- Algoritmy boli experimentálne overené na reálnom systéme
- Pri MRAC sa ukázalo, že rôzne platformy môžu dať rozdielne výsledky aj pri rovnakej logike algoritmu

Ďakujeme za pozornosť

1. Pri logovaní dát v PLC do polí sa index postupne inkrementuje, pričom nie je explicitne riešené nulovanie uložených dát. Ako by ste v praxi riešili opakované merania bez potreby zmeny režimu CPU?

```
IF NOT #Full AND #Enable THEN
  IF #idx <= 9999 THEN
    #TimeLog_s[#idx] := #CurrentTimeSec;
    #ULOG[#idx] := #u;
    #YLOG[#idx] := #y;
    #RLOG[#idx] := #r;
    #YPBLOG[#idx] := #ypb;
    #idx := #idx + 1;
    #SampleCount := #idx;
  ELSE
    #Full := TRUE;
  END_IF;
END_IF;
```

Výpis kódu 7: Implementácia logovacieho bloku FB_PI_LOGGER

1. Reset logovacieho bloku

idx = 0, SampleCount = 0, Full = FALSE, LOGS[all] = 0;

2. Stavový automat

IDLE → RESET → MEASURE → DONE

3. Kruhový buffer + snapshot buffery

2. Na predmete Riadiace systémy ste sa stretli s inštrukciou PID_Compact, ktorá je dostupná v knižnici TIA Portal. Neuvažovali ste o porovnaní vlastnej implementácie aspoň s jedným existujúcim riešením (napr. PID_Compact)? Má PID_Compact v porovnaní s Vaším riešením nejakú nevýhodu? Stručne vysvetlite.

PID_Compact:

$$y = K_P \left[(b * w - x) + \frac{1}{T_I S} (w - x) + \frac{T_D S}{a * T_D S + 1} (c * w - x) \right] \implies K_P \left[(w - x) + \frac{1}{T_I S} (w - x) \right]$$

Teda: $K_{P_compact} = K_P$, $T_{I_compact} = K_P/K_I$, **ale:**

Vlastná implementácia

- + jednoduchšia štruktúra
- + menej výpočtov
- + transparentný algoritmus
- + ľahká úprava logiky
- + možný aj čistý I regulátor

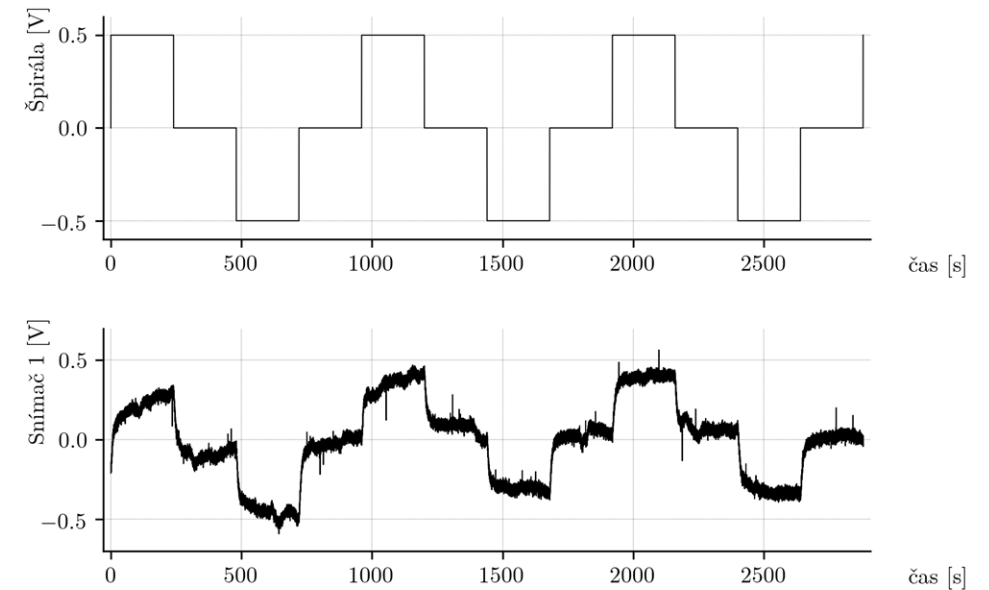
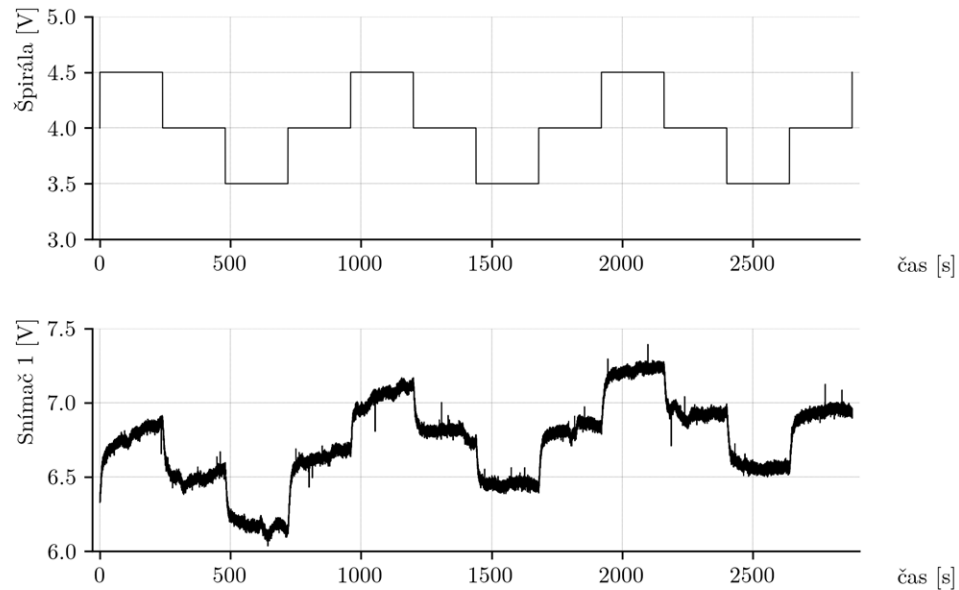
- menej funkcií
- bez komplexného anti-windup
- bez diagnostiky a autotuningu

PID_Compact

- + priemyselný štandard
- + diagnostika
- + autotuning
- + anti-windup
- + vhodný pre bežnú prax

- zložitejší blok
- menej transparentný vnútorný výpočet
- parametre sú viazané cez K_P
- proprietárnosť

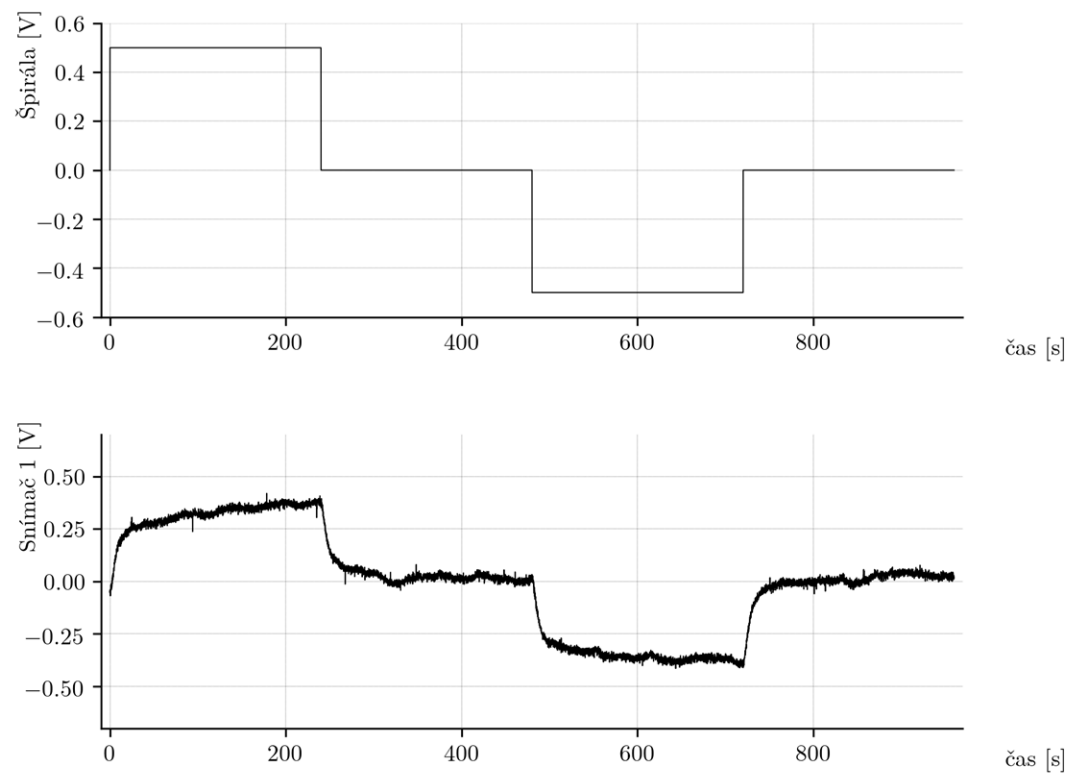
Merania na identifikáciu systému v pracovnom bode: Detrend



Obr. 10: Vstupný signál špirály a výstup senzora 1 pri skokovej zmene

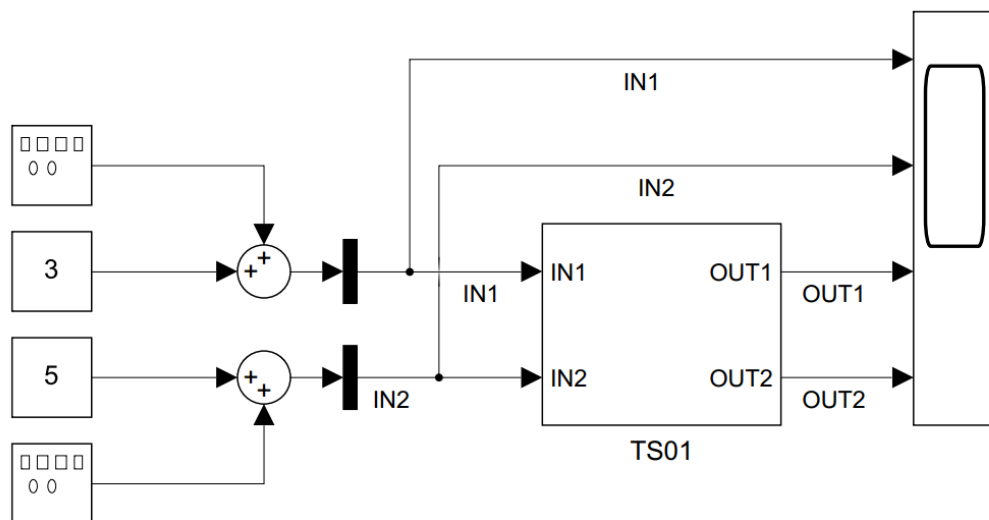
Obr. 11: Normalizované a detrendované priebehy vstupného a výstupného signálu

Merania na identifikáciu systému v pracovnom bode: Prechodová charakteristika

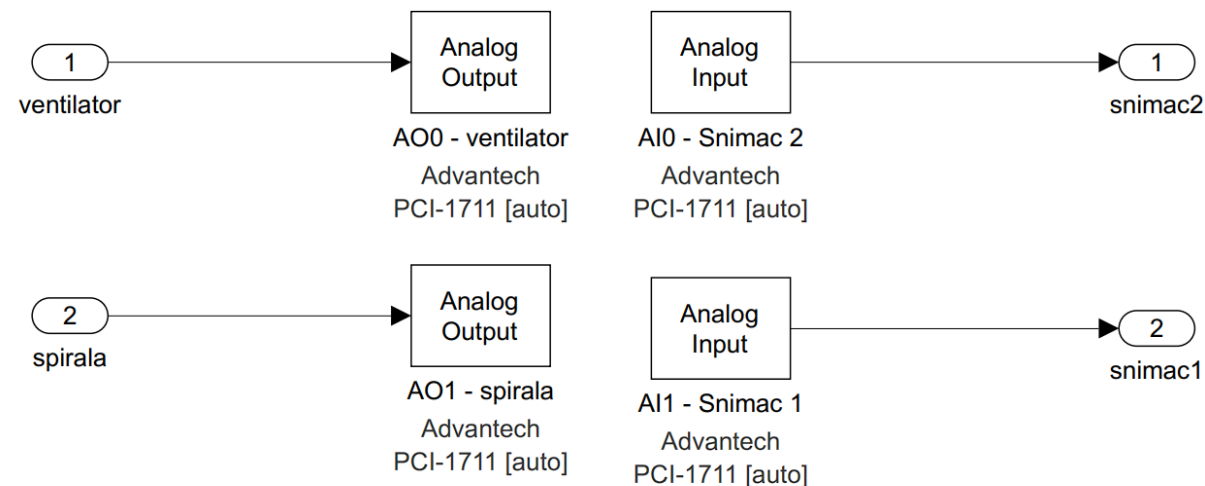


Obr. 12: Priemerná odozva systému TS05 v pracovnom bode 4 V

Pôvodná komunikácia v prostredí MATLAB/Simulink

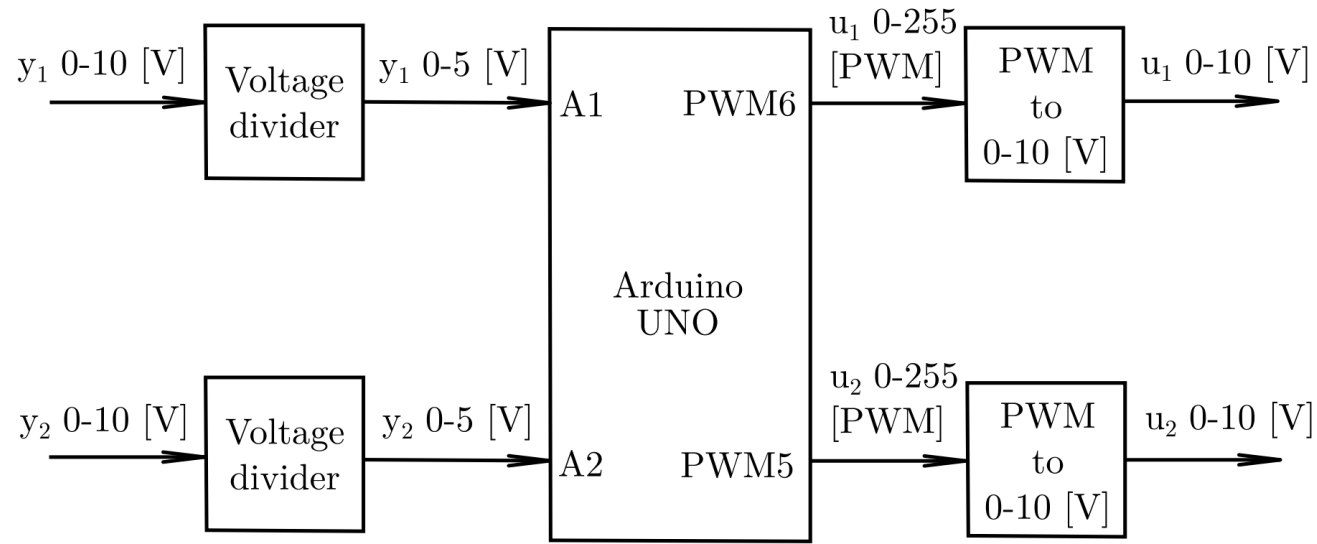


Obr. 15: Základná Simulinková schéma pre spustenie komunikácie so systémom TS05



Obr. 16: Vnútna časť Simulinkového bloku komunikácie so systémom TS05

Vlastná komunikácia v jazyku C++



Pin	Funkcia
1	Snímač 1
2	Snímač 2
6	Ventilátor
7	Špirála
8	GND

Tabuľka 2: Identifikované kontakty konektora systému TS05

Obr. 17: Schéma prepojenia systému TS05 s platformou Arduino UNO

Port Arduino	Signál	Reálny rozsah	Digitálny rozsah
A1	Snímač 1	0–10 V	0–1023
A2	Snímač 2	0–10 V	0–1023
PWM5	Ventilátor	0–10 V	0–255
PWM6	Špirála	0–10 V	0–255

Tabuľka 3: Mapovanie vstupov a výstupov medzi TS05 a Arduino UNO

$$U_{\text{out}} = U_{\text{in}} \frac{R_2}{R_1 + R_2}$$

$$U_{\text{out}} = \frac{U_{\text{in}}}{2}$$

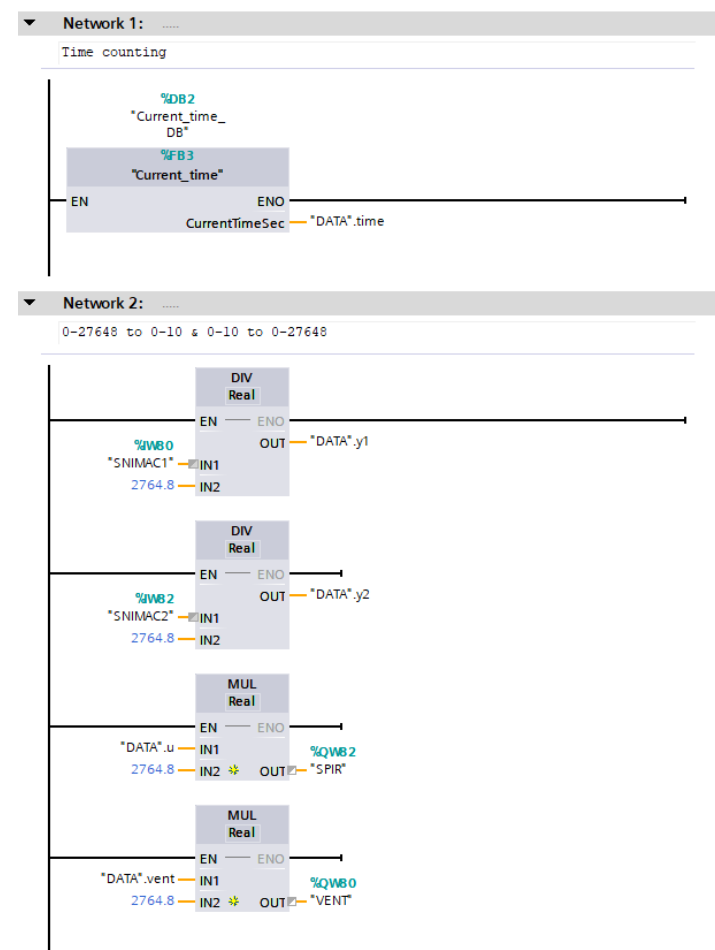
Komunikácia s programovateľným logickým automatom SIMATIC S7-1200 G2

Signál	Typ	Adresa
Ventilátor	Int	%QW80
Špirála	Int	%QW82
Snímač 1	Int	%IW80
Snímač 2	Int	%IW82

Tabuľka 8: Adresovanie analógových vstupov a výstupov PLC modulu pre systém TS05

$$U_{in} = \frac{INT}{2764.8}$$

$$INT = U_{out} \cdot 2764.8$$



Obr. 18: Servisná časť hlavného organizačného bloku PLC programu

Komunikácia s programovateľným logickým automatom SIMATIC S7-1200 G2

```
#retVal := RD_SYS_T(OUT => #currentTime);  
  
IF (NOT #started) AND (#retVal = 0) THEN  
    #startTime := #currentTime;  
    #started := TRUE;  
END_IF;  
  
IF #started AND (#retVal = 0) THEN  
    #elapsedTime := T_DIFF(IN1 := #currentTime, IN2 := #startTime);  
    #CurrentTimeSec := DINT_TO_LREAL(TIME_TO_DINT(#elapsedTime)) / 1000.0;  
ELSE  
    #CurrentTimeSec := 0.0;  
END_IF;
```

Výpis kódu 6: Implementácia funkčného bloku Current_time

```
IF NOT #Full AND #Enable THEN  
    IF #idx <= 9999 THEN  
        #TimeLog_s[#idx] := #CurrentTimeSec;  
        #ULOG[#idx] := #u;  
        #YLOG[#idx] := #y;  
        #RLOG[#idx] := #r;  
        #YPBLOG[#idx] := #ypb;  
        #idx := #idx + 1;  
        #SampleCount := #idx;  
    ELSE  
        #Full := TRUE;  
    END_IF;  
END_IF;
```

Výpis kódu 7: Implementácia logovacieho bloku FB_PI_LOGGER